used with C++20. But this time, this keyword is used to mark a declaration, a group of declarations, or another module exported by the current module. But the status of the previously unused and reserved keyword *register* remains the same in C++20 also. Special types of functions called *coroutines* that can have their execution suspended and resumed are implemented in C++20. The keywords *co_await*, *co_return* and *co_yield* are used to manipulate *coroutines*. A mechanism called *modules* that will help us divide large amount of code into logical sections was also introduced in C++20.

Being very recent, C++20 is not yet fully supported by compilers. The support for C++20 has started with g++ compiler version 8. Since this support is an ongoing process, it is better to use g++ compiler version 11 (the latest version as of April 2021). Since my preferred version is C++11, I never had to update version 7 of g++ in my system. The command 'g++ --version' will tell you the version of the g++ compiler in your system. If the version is older than version 8, then you won't be able to use the g++ option '-std=c++20'. But there is an easy way to overcome this problem. Since g++ is a part of the Ubuntu tool chain, you can install g++ version 11 in your system with the command 'sudo apt install g++-11'. The default version of g++ will still be your older version. Make sure that you use the command 'g++-11' instead of 'g++' when you need to use version 11, because the command 'g++' will only invoke the older version of g++. Now, consider the program *pgm5.cc* given below:

```
#include<iostream>
using namespace std;

consteval int cube(int x)
{
  return x*x*x;
}
int main()
{
     constexpr int n = cube(5);
     cout<<n<<endl;
     return 0;
}
```

The keyword *consteval* declares a function as an *immediate* function. An *immediate* function returns a constant value at compile time. This is especially useful for optimisation purposes. The compilation and execution of the program *pgm5.cc* conforming to the C++20 standard is given below:

```
>> g++-11 -std=c++20 pgm5.cc
>> ./a.out
125
```

C++23 standard

C++23 is the upcoming standard of C++. Some of the proposed features of C++23 include usage of a literal suffix for signed *size_t*, modified syntax for lambda functions, Standard Library support for *coroutines*, the creation of a modular Standard Library, etc. Experimental support for some of these proposed features of C++23 is already available in g++, including usage of a literal suffix for signed *size_t*, modified syntax for lambda functions, etc. The experimental support for C++23 only began with g++ version 11. Thus, we need g++-11 to use the option '=std=c++23'. Since we have already installed g++-11 in our system, we can directly use it. The compilation and execution of the program *pgm5.cc* conforming to the C++23 standard is given below:

```
>> g++-11 -std=c++23 pgm5.cc
>> ./a.out
125
```

Now it is time to end our discussion. I do agree that our treatment of different C++ standards was very limited. Moreover, some of the features added in the latest C++ standards are a bit tricky to understand. But the aim of the article is only to discuss the different standards themselves. Also, now we know which specific version of g++ may be used to compile programs conforming to a particular standard. Though the official ISO standardisation documents of C++ are not free (as in free beer), working drafts are freely available on the Internet. With the help of those documents, interested programmers can dig deeper into a particular C++ standard for which they have a liking.

But before we part ways, one question needs to be answered. What standard of C++ should a young student adopt? The choices include the most popular (C++98, at least in academia), the most stable and modern (C++17), and the most modern (C++20). Though it is a matter of personal choice, I would suggest the adoption of C++17 or C++11 over C++98. It is a fact that the changes brought in by C++20 are more profound when compared with C++17. But adoption of C++20, while awaiting full support from compilers, might be a bit risky at this stage. If you are familiar with C++ and planning to adopt C++11, I suggest the textbook 'The C++ Programming Language (4th Edition)'. If you are familiar with C++ and planning to adopt C++17, I suggest the text book 'A Tour of C++ (2nd Edition)'. Both the textbooks are written by Bjarne Stroustrup. Yes! Why learn from a disciple when you can learn from the master himself. END

By: Deepu Benson

The author is a free software enthusiast whose area of interest is theoretical computer science. The open source tools of his choice include ns-2 and ns-3. The author maintains a technical blog at *www.computingforbeginners.blogspot.in*.